OCC: SOFTWARE ENGINEER YEAR 2

SAQA ID: 119458

NQF LEVEL: 6

Software design and testing With C#

SDT621

Formative Assessment 2

2026

CTU training solutions | Private Tertiary Education

Table of Contents

DECLARATION OF AUTHENTICITY

I Stefan Venter (FULL NAME)

hereby declare that the contents of this assessment SDT621 FA 2

is entirely my own work, completed by me without any paraphrasing/copying, or presented as my own work accessed from any AI Apps, example ChatGPT, Co-pilot, Perplexity, or any other App.

I have read and understood the Examination Rules and Regulations.

Signature: 

Date: 08/05/2026

QUALIFICATION AND ASSESSMENT DETAILS

Programme Title	OCC: SOFTWARE ENGINEER
Module Title	Software design and testing C#
Module Code	SDT621
NQF Level	6
Assessment Type	Formative Assessment 2
Hand Out Date	13 / 0 4 / 2026
Hand in Date	08 / 05 / 2026
Total Marks	100
Assessment Developer	Lusukama Selemani
Internal Moderator	Faith Muwishi
Internal Moderator	Michelle Du Toit
External Moderator	

Student Name and Surname	Stefan Venter
Student Number	20240113

Instructions:

Please read the following instructions carefully before attempting this assessment:

- Read each task carefully and consider the mark allocation before responding.
- This assessment is based on an industry-aligned software testing and debugging scenario.
- You are required to demonstrate your ability to:
    - design test specifications
    - perform functional and non-functional testing
    - identify defects
    - debug source code
    - document testing activities
    - produce a structured testing report
- Apply appropriate software testing methodologies and techniques when completing the tasks.
- Ensure that all responses are clear, structured, and professionally documented.
- Where required, include corrected source code, test cases, defect logs, and screenshots as evidence.

Submission Requirements

For your final submission, you must include the following:

- Declaration of Authenticity
- Completed Answer Sheet (PDF format)
- Test Plan document
- Test Case Specification document
- Defect Log / Bug Report document
- Software Testing Report
- Corrected source code
- Visual Studio project files (compressed as a ZIP file)

Failure to submit all required components may result in loss of marks.

Evidence Collection Requirements

To support your submission, you must provide the following evidence:

- Screenshots showing execution of functional tests
- Screenshots showing non-functional testing activities (where applicable)
- Screenshots showing identified defects
- Screenshots showing corrected outputs after debugging

Each screenshot must:

- show the full screen
- clearly display the system date and time
- correspond to the task being performed

Important:

Do NOT submit screenshots of large sections of source code as evidence.

Screenshots must demonstrate testing execution evidence, not code duplication.

Debugging Evidence Requirements

Where debugging activities are required, you must:

Include the following for each defect identified:

- original faulty code
- description of the issue
- explanation of why the issue occurs
- corrected code snippet
- corrected program output screenshot

This demonstrates competency in:

Test or debug source code to ensure client needs are met (PM-04 outcome requirement).

Test Documentation Requirements

Your submission must include structured testing documentation such as:

Test Plan

Include:

- testing objectives
- scope of testing
- testing environment
- testing approach
- testing tools used

Test Cases

Each test case must include:

- test case ID
- module / feature tested
- input data
- expected result
- actual result
- pass / fail status

Minimum required:

10 structured professional test cases

Defect Reporting Requirements

You must produce a structured defect report containing:

- defect ID
- module affected
- description of defect
- severity level
- priority level
- status
- corrective action taken

Minimum required:

5 recorded defects

Software Testing Report Requirement

Your final submission must include a **Software Testing Report** containing:

- testing summary
- testing scope
- functional testing results
- non-functional testing results
- defects identified
- debugging corrections implemented
- recommendations
- deployment readiness decision

This report represents a key deliverable expected from a **Junior Software Tester in industry practice**.

GitHub Submission Requirement

You are required to:

- push your corrected solution code to GitHub
- ensure the repository contains the complete working project
- include test documentation where applicable
- share the GitHub repository link in your submission

Example:

https://github.com/your-username/project-name

Your repository must:

- build successfully
- contain corrected source code
- include testing evidence where applicable
- match the work submitted in your ZIP file

Failure to provide a working repository may result in marks being deducted.

Academic Integrity Requirement

All submitted work must be:

- your own original work
- aligned with the provided scenario
- supported by evidence of testing execution

Submitting copied work or incomplete testing documentation will result in penalties according to institutional assessment policy.

Assessment Outcomes

By completing this assessment successfully, you will demonstrate the ability to:

- explain software testing methodologies and techniques
- design functional test specifications
- design non-functional test specifications
- execute functional tests against requirements
- execute non-functional tests against requirements
- identify defects within source code
- debug source code errors
- record testing activities professionally
- produce a structured software testing report

Example of Evidence Collection for Submission PDF

Example:

Task 1 – Functional Testing Execution Evidence

Include:

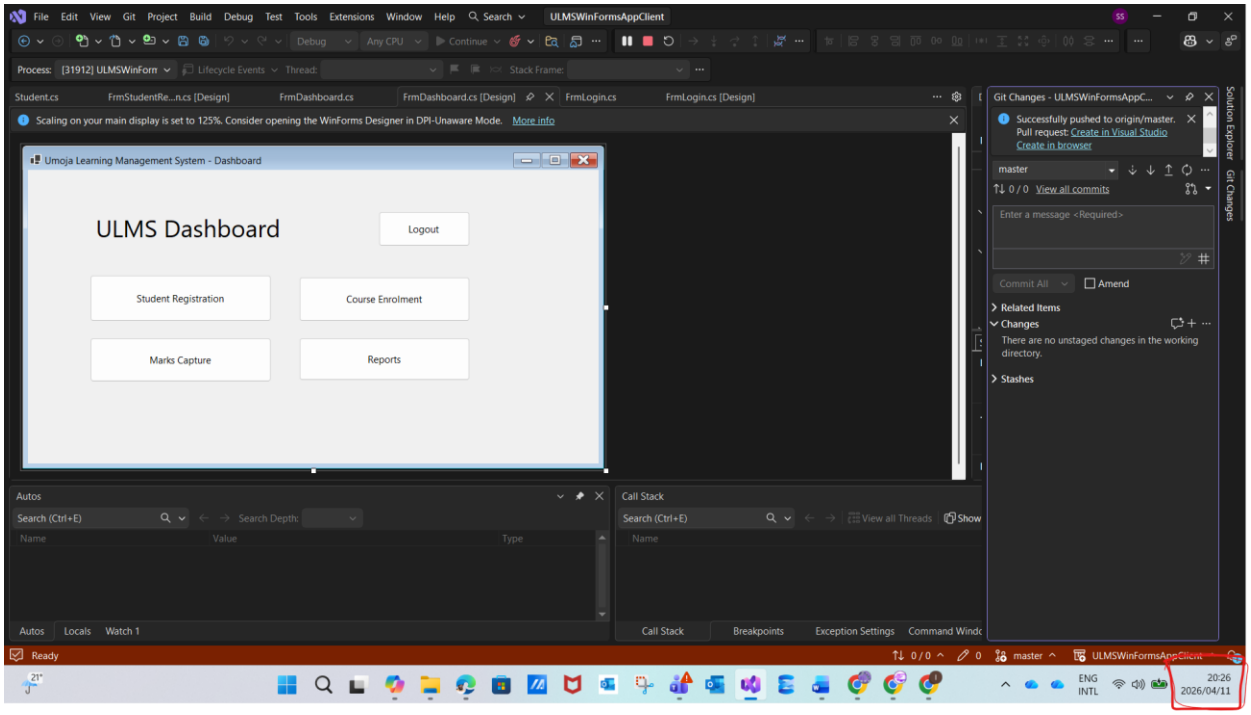
Screenshot showing:

- test execution
- expected result
- actual result
- pass/fail confirmation
- visible system date and time

Example format:

Ensure each screenshot clearly shows the full screen including the system date and time.

Question1 :



Ensure each screenshot shows the **full screen**, including the **system date and time**

CASE STUDY

Umoja Learning Management System (ULMS)

Umoja Group is launching a new cloud-enabled Learning Management System (ULMS) designed for:

- student registrations
- login authentication
- course enrolment
- marks capture
- report generation

However, before deployment, the organisation identified several issues:

- incorrect login validation behaviour
- inconsistent calculation of student averages
- incorrect course enrollment rules
- slow performance when loading reports
- missing error handling messages
- incomplete testing documentation

You are appointed as a Junior Software Tester

Your responsibility:

Perform structured testing and debugging activities to ensure the system meets client requirements before production deployment.

Clone the project here: https://github.com/lusuJR/ULMSWinFormsAppClient.git

PROJECT DELIVERABLE STRUCTURE

Learners must complete:

Deliverable	Evidence Type
Test Plan	Documentation
Test Cases	Documentation
Test Execution	Practical
Bug Identification	Technical
Debugging Evidence	Code
Test Report	Formal QA Report
Reflection	Professional Practice

TASK 1

SOFTWARE TESTING STRATEGY DESIGN

Marks: 20

Learners must develop a structured **Test Plan**

Include:

1.1 Testing Objectives

Define:

- system purpose
- testing goals
- scope of testing

1.2 Testing Types Identification

Explain testing approach for:

Testing Type	Example
Functional testing	login validation
Input validation	marks entry
Integration testing	course allocation
Performance testing	report loading
Usability testing	navigation
Security testing	authentication

1.3 Testing Environment Setup

Define:

- Hardware
- Software
- Browser / OS

Example Tools used :

- Visual Studio
- SQL Server
- Test data sets

TASK 2

TEST CASE DESIGN

Marks: 20

Design structured test cases for:

Feature	Required Test Cases
Student login validation	valid / invalid
Course enrollment	duplicates
Marks capture	numeric validation
Average calculation	boundary testing
Report generation	output correctness

Test case format must include:

Test Case ID	Description	Input	Expected Output	Actual Output	Status

Minimum:

10 professional test cases

TASK 3

FUNCTIONAL TEST EXECUTION

Marks: 15

Execute designed test cases and record results

Evidence required:

- Screenshot results
- Execution logs

Status classification:

Status
Pass
Fail
Blocked

TASK 4

NON-FUNCTIONAL TESTING

Marks: 15

Perform:

Type	Example
Performance testing	report loading time
Usability testing	navigation steps
Security testing	invalid login attempts
Reliability testing	repeated operations

Learners must:

- Describe method
- Record results
- Interpret findings

TASK 5

DEBUGGING SOURCE CODE

Marks: 15

Learners must identify and correct errors in supplied faulty modules

Example faulty logic areas:

- Incorrect login validation
- Incorrect average calculation
- Missing exception handling
- Incorrect conditional statements

Learners must provide:

Evidence
Original faulty code
Identified issue
Corrected version
Explanation

TASK 6

DEFECT REPORTING

Marks: 10

Learners must produce structured bug report table

Example format:

Bug ID	Module	Description	Severity	Priority	Status	Resolution

Minimum:

5 defects

TASK 7

SOFTWARE TESTING REPORT

Marks: 5

Final professional QA Report including:

Executive summary

Testing scope

Tools used

Defects identified

Corrections implemented

Recommendations

Deployment readiness decision

TOTAL MARKS ..... : _____ / 100

END OF ASSESSMENT

MARK ALLOCATION

Task	Description	Marks
Task 1	Test Plan Design	20
Task 2	Test Case Design	20
Task 3	Functional Testing Execution	15
Task 4	Non-Functional Testing	15
Task 5	Debugging Activities	15
Task 6	Defect Reporting	10
Task 7	Final Test Report	5
TOTAL		**100**

LEARNING OUTCOMES ALIGNMENT GRID

Mapped directly to QCTO Associated Assessment Criteria

Learning Outcome	Evidence Produced
Explain testing methodologies	Test plan
Design functional test specifications	Test cases
Design non-functional test specifications	Test cases
Execute functional tests	Execution logs
Execute non-functional tests	Performance/usability testing
Debug source code	corrected logic
Record testing activities	defect log
Produce testing report	QA report

INDUSTRY AUTHENTICITY FEATURES

This assessment simulates real roles:

Role Simulation
QA Tester
Software Validation Engineer
Test Analyst
Junior DevOps QA Support